GeoLifeSpirit — Firmware Config Sync Referans Kodu

Amaç: Cihaz, kullanıcının mobil uygulamadan seçtiği gönderim aralığına göre telemetri göndermelidir. Sabit aralıkta (8sn gibi) gönderim YAPILMAMALIDIR. Sunucudaki config'e uyulmalıdır.

Mantık Özeti

```
BOOT → Config Çek → reportIntervalSec al → Bu aralıkla gönder
        ↓
Her 5 döngüde bir config tekrar çek → configVersion değiştiyse güncelle
        ↓
Batarya düşükse → powerSaveThreshold'a göre aralığı artır
        ↓
Acil durum varsa → emergencyIntervalSec ile daha sık gönder
```

Config Endpoint

```
GET https://api.geolifespirit.com/v2/telemetry/config/<DEVICE_ID>
```

Auth header GEREKMEZ. Response:

```json
{
  "ok": true,
  "config": {
    "reportIntervalSec": 60,
    "emergencyEnabled": true,
    "emergencyIntervalSec": 10,
    "lowBatteryThreshold": 15,
    "powerSaveThreshold": 10,
    "configVersion": 3
  }
}
```

reportIntervalSec Değerleri

Kullanıcı mobil uygulamadan bu değerlerden birini seçer:

Değer	Anlamı	Not
`0`	Canlı mod (sürekli gönder)	Premium abonelik gerektirir
`10`	10 saniyede bir	Premium abonelik gerektirir
`30`	30 saniyede bir	Premium abonelik gerektirir
`60`	1 dakikada bir	Varsayılan
`300`	5 dakikada bir	
`900`	15 dakikada bir	
`1800`	30 dakikada bir	
`3600`	1 saatte bir	

Referans Firmware Kodu (C — ESP32/STM32 uyumlu pseudo-code)

```c
```

```c
/*************************************************************
 * GeoLifeSpirit — Firmware Telemetry & Config Sync
 *
 * Bu kod referans amaçlıdır. Donanım spesifik HTTP client,
 * JSON parser ve sleep fonksiyonları platforma göre
 * değiştirilmelidir.
 *************************************************************/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

/* ═══════════════════════════════════════════════
   KONFİGÜRASYON YAPISI
   ═══════════════════════════════════════════════ */

typedef struct {
    int  report_interval_sec;     // Sunucudan gelen gönderim aralığı (saniye)
    bool emergency_enabled;       // Acil durum modu aktif mi
    int  emergency_interval_sec;  // Acil durumda gönderim aralığı
    int  low_battery_threshold;   // Düşük batarya eşiği (%)
    int  power_save_threshold;    // Güç tasarrufu eşiği (%)
    int  config_version;          // Config versiyon numarası
} DeviceConfig;

/* ═══════════════════════════════════════════════
   GLOBAL DEĞİŞKENLER
   ═══════════════════════════════════════════════ */

static DeviceConfig g_config = {
    .report_interval_sec    = 60,    // Varsayılan: 1 dakika
    .emergency_enabled      = false,
    .emergency_interval_sec = 10,
    .low_battery_threshold  = 15,
    .power_save_threshold   = 10,
    .config_version         = 0,
};

static int  g_cycle_count     = 0;        // Döngü sayacı
static int  g_event_counter   = 0;        // eventId için monotonic counter
static bool g_emergency_active = false;   // Acil durum şu an aktif mi

#define CONFIG_SYNC_EVERY_N_CYCLES  5     // Her 5 döngüde bir config sync
#define LIVE_MODE_INTERVAL_MS       1000  // Canlı mod: 1 saniye (reportIntervalSec=
#define POWER_SAVE_MULTIPLIER       3     // Güç tasarrufunda interval 3x artırılır
```

```c
#define MAX_RETRY_COUNT             3     // Gönderim başarısız olursa max retry

/* ═══════════════════════════════════════════════
   PLATFORM-SPECIFIC FONKSIYON DEKLERASYONLARİ
   (Bu fonksiyonları kendi platformunuza göre implemente edin)
   ═══════════════════════════════════════════════ */

// HTTP GET isteği gönder, response body'yi buffer'a yaz
extern int http_get(const char* url, char* response_buf, int buf_size);

// HTTP POST isteği gönder
extern int http_post(const char* url, const char* headers,
                     const char* body, char* response_buf, int buf_size);

// JSON parse helper'lar
extern int    json_get_int(const char* json, const char* key);
extern bool   json_get_bool(const char* json, const char* key);
extern double json_get_double(const char* json, const char* key);

// Sensör okuma fonksiyonları
extern double sensor_read_battery_percentage(void);
extern bool   sensor_is_charging(void);
extern double sensor_read_latitude(void);
extern double sensor_read_longitude(void);
extern double sensor_read_altitude(void);
extern double sensor_read_speed(void);
extern int    sensor_read_direction(void);
extern int    sensor_read_satellites(void);
extern int    sensor_read_heart_rate(void);
extern int    sensor_read_respiration_rate(void);
extern int    sensor_read_step_count(void);
extern bool   sensor_is_collar_removed(void);
extern bool   sensor_is_sos_pressed(void);
extern bool   sensor_has_gps_fix(void);

// Sleep fonksiyonu (milisaniye)
extern void platform_sleep_ms(unsigned long ms);

// Log fonksiyonu
extern void debug_log(const char* fmt, ...);

/* ═══════════════════════════════════════════════
   1. CONFIG SYNC — Sunucudan konfigürasyon çek
   ═══════════════════════════════════════════════ */

/**
 * Sunucudan config çeker ve g_config'i günceller.
```

```
 * configVersion değişmediyse bir şey yapmaz.
 *
 * @return true: başarılı, false: başarısız
 */
bool sync_config(const char* device_id) {
    char url[256];
    char response[2048];

    snprintf(url, sizeof(url),
        "https://api.geolifespirit.com/v2/telemetry/config/%s",
        device_id);

    int status = http_get(url, response, sizeof(response));
    if (status != 200) {
        debug_log("[CONFIG] Sync basarisiz, HTTP %d", status);
        return false;
    }

    // configVersion kontrol et — değişmediyse güncelleme yapma
    int new_version = json_get_int(response, "config.configVersion");
    if (new_version <= g_config.config_version) {
        debug_log("[CONFIG] Versiyon ayni (%d), degisiklik yok", new_version);
        return true;
    }

    // Yeni config'i uygula
    int new_interval = json_get_int(response, "config.reportIntervalSec");

    // Geçerlilik kontrolü: sadece izin verilen değerler
    if (new_interval == 0   || new_interval == 10   || new_interval == 30   ||
        new_interval == 60  || new_interval == 300  || new_interval == 900  ||
        new_interval == 1800 || new_interval == 3600) {
        g_config.report_interval_sec = new_interval;
    } else {
        debug_log("[CONFIG] Gecersiz interval: %d, varsayilan 60 kullaniliyor", new_
        g_config.report_interval_sec = 60;
    }

    g_config.emergency_enabled      = json_get_bool(response, "config.emergencyEnabl
    g_config.emergency_interval_sec = json_get_int(response, "config.emergencyInterv
    g_config.low_battery_threshold  = json_get_int(response, "config.lowBatteryThres
    g_config.power_save_threshold   = json_get_int(response, "config.powerSaveThresh
    g_config.config_version         = new_version;

    debug_log("[CONFIG] Guncellendi v%d: interval=%ds, emergency=%s",
        new_version,
        g_config.report_interval_sec,
```

```
        g_config.emergency_enabled ? "ON" : "OFF");

    return true;
}

/* ═══════════════════════════════════════════════
   2. AKTİF GÖNDERİM ARALIĞINI HESAPLA
   ═══════════════════════════════════════════════ */

/**
 * Mevcut duruma göre kaç saniye beklemesi gerektiğini döndürür.
 *
 * Öncelik sırası:
 *   1. Acil durum aktif   → emergencyIntervalSec (en kısa)
 *   2. Canlı mod (0)      → 1 saniye
 *   3. Güç tasarrufu       → reportIntervalSec × 3
 *   4. Normal mod          → reportIntervalSec
 */
int calculate_sleep_seconds(double battery_percentage) {

    // 1. Acil durum modu: en yüksek öncelik
    if (g_emergency_active && g_config.emergency_enabled) {
        int interval = g_config.emergency_interval_sec;
        if (interval <= 0) interval = 10;
        debug_log("[SLEEP] Acil durum modu: %d saniye", interval);
        return interval;
    }

    // 2. Canlı mod (reportIntervalSec == 0)
    if (g_config.report_interval_sec == 0) {
        debug_log("[SLEEP] Canli mod: 1 saniye");
        return 1;
    }

    // 3. Güç tasarrufu: batarya eşiğin altındaysa interval'i artır
    if (battery_percentage <= (double)g_config.power_save_threshold) {
        int interval = g_config.report_interval_sec * POWER_SAVE_MULTIPLIER;
        debug_log("[SLEEP] Guc tasarrufu (batarya %.1f%%): %d saniye",
            battery_percentage, interval);
        return interval;
    }

    // 4. Normal mod: sunucudan gelen interval
    debug_log("[SLEEP] Normal mod: %d saniye", g_config.report_interval_sec);
    return g_config.report_interval_sec;
}
```

```c
/* ═══════════════════════════════════════════════
   3. ACİL DURUM TESPİTİ (firmware tarafı)
   ═══════════════════════════════════════════════ */

/**
 * Sensör verilerine bakarak acil durum olup olmadığını belirler.
 * Acil durumda gönderim aralığı emergencyIntervalSec'e düşer.
 */
bool check_emergency_condition(void) {
    // Tasma çıkarıldı
    if (sensor_is_collar_removed()) return true;

    // SOS butonu basıldı
    if (sensor_is_sos_pressed()) return true;

    // Kritik batarya (firmware tarafında da kontrol)
    double battery = sensor_read_battery_percentage();
    if (battery <= 5.0) return true;

    return false;
}

/* ═══════════════════════════════════════════════
   4. TELEMETRİ PAKETİ OLUŞTUR & GÖNDER
   ═══════════════════════════════════════════════ */

/**
 * Sensör verilerini okur, JSON paketi oluşturur ve sunucuya gönderir.
 * Başarısız olursa exponential backoff ile retry yapar.
 *
 * @return true: başarılı gönderim, false: tüm retry'lar başarısız
 */
bool send_telemetry(const char* device_id, const char* device_key) {
    char body[4096];
    char response[512];

    // Sensör verilerini oku
    double battery     = sensor_read_battery_percentage();
    bool   charging    = sensor_is_charging();
    bool   has_gps     = sensor_has_gps_fix();
    int    heart_rate  = sensor_read_heart_rate();
    int    resp_rate   = sensor_read_respiration_rate();
    int    steps       = sensor_read_step_count();
    bool   collar_off  = sensor_is_collar_removed();
    bool   sos         = sensor_is_sos_pressed();

    // Benzersiz eventId oluştur
```

```
g_event_counter++;

// ── JSON body oluştur ──
// NOT: batteryPercentage ve isCharging → TOP LEVEL (payload dışında!)

int offset = snprintf(body, sizeof(body),
    "{"
    "\"eventId\":\"%s-%08d\","
    "\"batteryPercentage\":%.1f,"
    "\"isCharging\":%s,"
    "\"payload\":{",
    device_id,                          // eventId prefix: deviceId + counter
    g_event_counter,
    battery,
    charging ? "true" : "false"
);

// ── payload.location (GPS fix varsa) ──
if (has_gps) {
    offset += snprintf(body + offset, sizeof(body) - offset,
        "\"location\":{"
        "\"latitude\":%.6f,"
        "\"longitude\":%.6f,"
        "\"altitude\":%.1f,"
        "\"speed\":%.1f,"
        "\"direction\":%d,"
        "\"satellites\":%d"
        "},",
        sensor_read_latitude(),
        sensor_read_longitude(),
        sensor_read_altitude(),
        sensor_read_speed(),
        sensor_read_direction(),
        sensor_read_satellites()
    );
}
// GPS fix yoksa location bölümü HİÇ GÖNDERİLMEZ

// ── payload.health ──
offset += snprintf(body + offset, sizeof(body) - offset,
    "\"health\":{"
    "\"heartRate\":%d,"
    "\"respirationRate\":%d"
    "},",
    heart_rate,
    resp_rate
);
```

```
// ── payload.activity ──
// (Bu bölümü kendi aktivite takip modülünüze göre doldurun)
offset += snprintf(body + offset, sizeof(body) - offset,
    "\"activity\":{"
    "\"currentState\":\"resting\","
    "\"totalStepCount\":%d,"
    "\"caloriesBurned\":0,"
    "\"walking\":{\"stepCount\":0,\"distanceCovered\":0,\"duration\":0,\"isActiv
    "\"running\":{\"stepCount\":0,\"distanceCovered\":0,\"duration\":0,\"isActiv
    "\"sleeping\":{\"duration\":0,\"isActive\":false,\"deepSleep\":false,\"lastM
    "\"resting\":{\"duration\":0,\"isActive\":true,\"lastMovement\":null}"
    "},",
    steps
);

// ── payload.collarStatus ──
if (collar_off) {
    offset += snprintf(body + offset, sizeof(body) - offset,
        "\"collarStatus\":{\"isRemoved\":true,\"removedAt\":null},"
    );
} else {
    offset += snprintf(body + offset, sizeof(body) - offset,
        "\"collarStatus\":{\"isRemoved\":false,\"removedAt\":null},"
    );
}

// ── payload.connectivity ──
offset += snprintf(body + offset, sizeof(body) - offset,
    "\"connectivity\":{"
    "\"gpsStatus\":%s,"
    "\"simCardStatus\":true,"
    "\"bluetoothStatus\":false,"
    "\"gpsSignalStrength\":%d,"
    "\"simCardSignalStrength\":%d"
    "},",
    has_gps ? "true" : "false",
    has_gps ? sensor_read_satellites() * 12 : 0,  // Yaklaşık sinyal kalitesi
    70  // Modem RSSI'dan hesaplayın
);

// ── payload.deviceInfo ──
offset += snprintf(body + offset, sizeof(body) - offset,
    "\"deviceInfo\":{"
    "\"imei\":\"%s\","
    "\"simIccid\":\"%s\","
    "\"phoneNumber\":\"%s\","
```

```
    "\"operatorName\":\"%s\","
    "\"firmwareVersion\":\"%s\""
    "},",
    get_imei(),           // Platform fonksiyonu
    get_sim_iccid(),      // Platform fonksiyonu
    get_phone_number(),   // Platform fonksiyonu
    get_operator_name(),  // Platform fonksiyonu
    FIRMWARE_VERSION      // Sabit tanım
);

// ── payload.sos ──
offset += snprintf(body + offset, sizeof(body) - offset,
    "\"sos\":%s",
    sos ? "true" : "false"
);

// JSON'u kapat
offset += snprintf(body + offset, sizeof(body) - offset, "}}");

// ── HTTP headers ──
char headers[512];
snprintf(headers, sizeof(headers),
    "Content-Type: application/json\r\n"
    "X-Device-Id: %s\r\n"
    "X-Device-Key: %s",
    device_id,
    device_key
);

// ── Gönderim + Retry ──
for (int attempt = 0; attempt < MAX_RETRY_COUNT; attempt++) {
    int status = http_post(
        "https://api.geolifespirit.com/v2/telemetry/ingest",
        headers, body, response, sizeof(response)
    );

    if (status == 200) {
        debug_log("[SEND] Basarili (attempt %d)", attempt + 1);
        return true;
    }

    if (status == 401 || status == 400) {
        // Auth veya format hatası — retry anlamsız
        debug_log("[SEND] Kalici hata HTTP %d, retry yapilmiyor", status);
        return false;
    }
```

```
        if (status == 429) {
            // Rate limit — 60 saniye bekle
            debug_log("[SEND] Rate limit, 60 saniye bekleniyor");
            platform_sleep_ms(60000);
            continue;
        }

        // 5xx veya network hatası — exponential backoff
        int wait_ms = (1 << attempt) * 5000;  // 5s, 10s, 20s
        debug_log("[SEND] Hata HTTP %d, %d ms sonra retry", status, wait_ms);
        platform_sleep_ms(wait_ms);
    }

    debug_log("[SEND] Tum retry'lar basarisiz");
    return false;
}

/* ═══════════════════════════════════════════════
   5. ANA DÖNGÜ
   ═══════════════════════════════════════════════ */

/**
 * Firmware ana döngüsü.
 *
 * AKIŞ:
 *   Boot → Config sync → Sensör oku → Gönder → Hesaplanan süre kadar uyu → Tekrarla
 *
 * ASLA sabit aralıkta gönderim yapma!
 * HER ZAMAN sunucudaki reportIntervalSec değerine uy.
 */
void telemetry_main_loop(const char* device_id, const char* device_key) {

    // ── BOOT: İlk config sync ──
    debug_log("=== GLS Firmware Baslatiliyor ===");
    sync_config(device_id);
    debug_log("[BOOT] reportIntervalSec = %d", g_config.report_interval_sec);

    // ── SONSUZ DÖNGÜ ──
    while (1) {
        g_cycle_count++;

        // ── Adım 1: Periyodik config sync (her N döngüde bir) ──
        if (g_cycle_count % CONFIG_SYNC_EVERY_N_CYCLES == 0) {
            debug_log("[CYCLE %d] Config sync yapiliyor...", g_cycle_count);
            sync_config(device_id);
        }
```

```
        // ── Adım 2: Acil durum kontrolü ──
        bool was_emergency = g_emergency_active;
        g_emergency_active = check_emergency_condition();

        if (g_emergency_active && !was_emergency) {
            debug_log("[EMERGENCY] Acil durum tespit edildi! Interval dusuruldu.");
        } else if (!g_emergency_active && was_emergency) {
            debug_log("[EMERGENCY] Acil durum sona erdi, normal moda donuluyor.");
        }

        // ── Adım 3: Telemetri gönder ──
        send_telemetry(device_id, device_key);

        // ── Adım 4: Aktif interval'e göre uyu ──
        double battery = sensor_read_battery_percentage();
        int sleep_sec = calculate_sleep_seconds(battery);

        debug_log("[CYCLE %d] %d saniye uyku...\n", g_cycle_count, sleep_sec);

        if (sleep_sec <= 0) sleep_sec = 1;  // Minimum 1 saniye
        platform_sleep_ms((unsigned long)sleep_sec * 1000);
    }
}

/* ═══════════════════════════════════════════
   KULLANIM
   ═══════════════════════════════════════════ */

/*
 * main() veya app_main() içinden çağırın:
 *
 *   #define DEVICE_ID   "315462c3-0be3-56ef"
 *   #define DEVICE_KEY  "TwyT_FuJaB2oZLl5KvtFC0Zmrguf77NQprDoBrI08RM"
 *
 *   void app_main() {
 *       // WiFi/LTE bağlantısını kur
 *       // ...
 *       telemetry_main_loop(DEVICE_ID, DEVICE_KEY);
 *   }
 */
```

Akış Diyagramı

```
        |  CIHAZ BOOT  |
        └──────┬───────┘
               │
        ┌──────▼───────┐
        | Config Sync  | GET /v2/telemetry/config/<id>
        | reportInterval|
        | = 60 (örnek) |
        └──────┬───────┘
               │
┌──────────────▼──────────────┐
|    ANA DÖNGÜ BAŞLA          |
└──────────────┬──────────────┘
               │
┌──────────────▼──────────────┐
|  Her 5 döngüde bir          |
|  config tekrar çek          |──→ configVersion değişti mi?
|  (configVersion kontrol)|     Evet → interval güncelle
└──────────────┬──────────────┘     Hayır → devam
               │
┌──────────────▼──────────────┐
|  Sensörleri oku             |
|  GPS, IMU, kalp, batarya|
└──────────────┬──────────────┘
               │
┌──────────────▼──────────────┐
|  Telemetri gönder           | POST /v2/telemetry/ingest
|  (retry mekanizmasıyla)     |
└──────────────┬──────────────┘
               │
┌──────────────▼──────────────┐
|  Bekleme süresini hesapla|
├─────────────────────────────┤
| Acil durum?                 |
|  → emergencyIntervalSec     | (ör: 10s)
| Canlı mod (0)?              |
|  → 1 saniye                 |
| Güç tasarrufu?              |
|  → interval × 3            | (ör: 60×3 = 180s)
| Normal?                     |
|  → reportIntervalSec        | (ör: 60s)
└──────────────┬──────────────┘
               │
┌──────────────▼──────────────┐
|  sleep(hesaplanan_süre)     |
└──────────────┬──────────────┘
```

```
│
└──→ DÖNGÜ BAŞINA DÖN
```

Sık Yapılan Hatalar

Hata	Doğrusu
❌ Sabit 8 saniyede gönderim	✅ `reportIntervalSec` değerine göre gönder
❌ Config'i hiç çekmemek	✅ Boot'ta + her 5 döngüde bir `GET /config/<id>` çek
❌ configVersion kontrol etmemek	✅ Sadece version değiştiyse config uygula
❌ batteryPercentage'ı payload içinde göndermek	✅ JSON root seviyesinde gönder
❌ GPS fix yokken location:{latitude:0, longitude:0} göndermek	✅ GPS fix yoksa location bölümünü hiç gönderme
❌ Süreleri milisaniye göndermek	✅ duration alanları saniye cinsindendir
❌ Her istekte config çekmek	✅ Her 5 döngüde bir yeterli, gereksiz trafik yapma
❌ 429 hatasında hemen retry	✅ 60 saniye bekle, sonra dene
❌ 401 hatasında retry	✅ Auth hatası kalıcı, retry yapma, logla